Multi-Source Job Market Data Ingestion Engine

Comprehensive Code Audit & System Architecture Documentation

Course: DATA 607 – Data Acquisition and Management
Repository: `data_ingestion`
Domain: Job Board Telemetry, Normalization & Relational ETL
Target Storage: Local SQLite (`jobs.db`) / Cloud PostgreSQL (`Neon`)
Execution Engine: Windows Task Scheduler / Python 3.11 Orchestrator
Date: October 2026 | Version 1.0.0

Production-Grade Software Architecture & Engineering Audit

Contents

1 Repository & Directory Walkthrough **2**
1.1 Directory Tree Overview . . . 2
1.2 Folder-by-Folder Responsibilities . . . 2
1.3 File-by-File Breakdown . . . 2
1.3.1 Configuration and Scripts . . . 2
1.3.2 Orchestration & Core Engine . . . 3
1.3.3 Fetchers (`src/fetchers/`) . . . 3
1.3.4 Parsers (`src/parsers/`) . . . 4
1.3.5 Database Loaders (`src/db/`) . . . 4

2 End-to-End Pipeline Architecture **5**
2.1 Architecture Diagram . . . 5
2.2 Data Lifecycle Stages . . . 5

3 Code Section & Logic Breakdown **7**
3.1 Logical Section 1: Orchestration & Configuration . . . 7
3.2 Logical Section 2: HTTP Infrastructure & Politeness . . . 7
3.3 Logical Section 3: Source Extractors (Fetchers) . . . 7
3.4 Logical Section 4: Data Normalization & Quality Validation . . . 8
3.5 Logical Section 5: Relational Persistence & Multi-Backend Loading . . . 8

4 Configuration, Dependencies & Runbook **9**
4.1 Environment Variables & Secrets . . . 9
4.2 External Dependencies . . . 9
4.3 Operational Runbook & CLI Reference . . . 9
4.3.1 Pre-flight Health Audit . . . 9
4.3.2 Standard Local Ingestion (SQLite) . . . 9
4.3.3 Production Ingestion (PostgreSQL / Neon) . . . 9
4.3.4 Automated Windows Task Scheduler Execution . . . 9
4.3.5 Test Suite Execution . . . 10

1 Repository & Directory Walkthrough

1.1 Directory Tree Overview

The repository follows a clean, modular structure separating configuration, extraction clients (fetchers), record transformation (parsers), relational database persistence (loaders), and automated scheduled task runners.

```
data_ingestion/
|-- config/
|   `-- sources.yaml                # Declarative endpoint, rate limit & schema configs
|-- data/
|   |-- db/
|   |   `-- jobs.db                 # Local SQLite database target
|   `-- raw/                        # Date-partitioned raw payload landing zone
|       |-- adzuna/<date>/data.json
|       |-- arbeitnow/<date>/data.json
|       |-- himalayas/<date>/data.json
|       |-- hn_hiring/<date>/data.json
|       |-- jobicy/<date>/data.json
|       |-- remoteok/<date>/data.json
|       |-- usajobs/<date>/data.json
|       `-- wwr/<date>/data.json
|-- logs/
|   |-- pipeline_YYYY-MM-DD.log     # Daily run logs and load metrics
|   `-- source_health_YYYY-MM-DD.json # Pre-flight diagnostic telemetry
|-- scripts/
|   |-- check_sources.py            # Diagnostic liveness, schema & robots.txt auditor
|   `-- run_daily.ps1               # Scheduled Task PowerShell wrapper
|-- src/
|   |-- pipeline.py                 # Core CLI orchestrator
|   |-- db/
|   |   |-- __init__.py             # DB module init
|   |   |-- load.py                 # SQLite idempotent persistence backend
|   |   |-- load_postgres.py        # PostgreSQL (Neon) persistence backend
|   |   |-- schema.sql              # Normalized SQLite DDL schema
|   |   `-- schema_postgres.sql     # Normalized PostgreSQL DDL schema
|   |-- fetchers/                   # Source extraction clients (HTTP/RSS/Algolia)
|   |   |-- __init__.py
|   |   |-- base.py                 # Throttling, retry/backoff & User-Agent plumbing
|   |   |-- adzuna.py, arbeitnow.py, himalayas.py, hn_hiring.py
|   |   `-- jobicy.py, remoteok.py, usajobs.py, wwr.py
|   `-- parsers/                    # Field normalization & quality validation
|       |-- __init__.py             # Canonical ROW_FIELDS and validator
|       |-- adzuna.py, arbeitnow.py, himalayas.py, hn_hiring.py
|       `-- jobicy.py, remoteok.py, usajobs.py, wwr.py
|-- tests/
|   |-- conftest.py, test_db_load.py, test_parsers.py
|   |-- test_rate_limit.py, test_retry.py
|   `-- fixtures/                   # Offline test payload snapshots
`-- requirements.txt                # Third-party Python dependencies
```

Listing 1: Hierarchical Repository Layout

1.2 Folder-by-Folder Responsibilities

1.3 File-by-File Breakdown

1.3.1 Configuration and Scripts

- **`config/sources.yaml`:** Source configuration manifest. Declares endpoint URLs, rate limits, access types, credential environment variables, and pre-parse required fields for 8 sources.
- **`scripts/check_sources.py`:** Standalone pre-flight auditor. Assesses API liveness, measures response latency, validates schema contracts against live payloads, checks `robots.txt` compliance, and writes telemetry to `logs/source_health_<date>.json`.
- **`scripts/run_daily.ps1`:** Production PowerShell task runner. Triggered by Windows Task Scheduler, invokes the pipeline orchestrator, logs execution output, and preserves exit codes.

Folder Path	Role within the Ingestion Lifecycle
`config/`	**Configuration Layer:** Houses declarative YAML files specifying ingestion endpoints, rate-limiting constraints, access credentials, and expected schema contracts.
`scripts/`	**Operations & Automation:** Contains operational automation utilities, including pre-flight source health audits and scheduled execution wrappers for Windows Task Scheduler.
`src/`	**Core Pipeline Package:** Contains the central CLI pipeline orchestrator coordinating extraction, disk staging, normalization, validation, and multi-backend loading.
`src/fetchers/`	**Extraction Layer:** Encapsulates protocol-specific network communication (REST, RSS XML, Algolia API) with respectful monotonic throttling and Tenacity exponential backoff.
`src/parsers/`	**Transformation & Quality Layer:** Maps heterogeneous raw schemas to a uniform 13-column relational contract and validates data types and URL integrity.
`src/db/`	**Persistence Layer:** Implements 3NF normalized DDL schemas and idempotent upsert drivers for both SQLite and PostgreSQL (Neon).
`data/raw/`	**Data Lake / Raw Staging:** Immutable date-partitioned landing zone where verbatim JSON/XML API responses are snapshotted prior to transformation.
`data/db/`	**Storage Target:** Dedicated directory storing the active SQLite database file (`jobs.db`).
`logs/`	**Telemetry & Observability:** Centralized log directory recording execution timestamps, pipeline summary tables, and JSON health reports.
`tests/`	**Quality Assurance:** Unit and integration test suites validating parsers, rate limiting, retries, and database idempotency against recorded fixtures.

Table 1: System Directory Responsibilities

1.3.2 Orchestration & Core Engine

- **`src/pipeline.py`:** Master orchestrator. Iterates over active sources, coordinates fetchers, writes raw payloads to disk, executes parsers and validation, and commits batches to the selected database backend. Entry functions: `main()`, `run_source()`.

1.3.3 Fetchers (`src/fetchers/`)

- **`base.py`:** Shared networking infrastructure. Provides monotonic interval throttling (`throttle()`), project-identified User-Agent generation (`build_headers()`), and Tenacity exponential retries (`get()`) targeting HTTP 429 and 5xx errors.
- **`remoteok.py`:** RemoteOK REST client. Fetches complete unpaginated listings, filtering out the initial metadata notice.
- **`arbeitnow.py`:** Arbeitnow REST client. Retrieves active postings under the `"data"` response key.
- **`jobicy.py`:** Jobicy API v2 client. Implements sample queries (`count=5`) and batch extractions (`count=50`).
- **`himalayas.py`:** Himalayas REST client. Executes batch queries with parameterized limits (`limit=100`).
- **`wwr.py`:** We Work Remotely RSS client. Downloads XML feeds and parses feed entries using `feedparser`.
- **`hn_hiring.py`:** Hacker News Algolia client. Dynamically resolves the latest "Ask HN: Who is hiring" monthly thread ID, then fetches child comments (up to 1,000 hits).
- **`adzuna.py`:** Authenticated Adzuna API client. Verifies `ADZUNA_APP_ID` and `ADZUNA_APP_KEY` to fetch US data science postings.
- **`usajobs.py`:** Authenticated USAJobs REST client. Utilizes `USAJOBS_API_KEY` and `USAJOBS_EMAIL`, flattening nested `MatchedObjectDescriptor` structures.

1.3.4 Parsers (`src/parsers/`)

- **`__init__.py`:** Schema contract definition. Declares `ROW_FIELDS`, implements `empty_row()`, and enforces pre-load validation (`validate_row()`) checking mandatory keys and URL structure.
- **Source Parsers (`remoteok.py`, `arbeitnow.py`, etc.):** Map source-specific attributes to the uniform 13-field record. Converts Unix epoch times to ISO UTC strings (`arbeitnow`), splits composite company-title strings (`wwr`), stores free-text comments verbatim without speculative inference (`hn_hiring`), and safely parses numerical salary bounds (`usajobs`).

1.3.5 Database Loaders (`src/db/`)

- **`schema.sql` & `schema_postgres.sql`:** Normalized 3NF DDL definitions for `sources`, `companies`, `jobs`, `tags`, and `job_tags`.
- **`load.py`:** SQLite persistence driver. Connects, applies DDL migrations, and performs idempotent upserts using `ON CONFLICT(source_id, external_id) DO UPDATE`.
- **`load_postgres.py`:** PostgreSQL/Neon persistence driver. Implements identical idempotent loading semantics using `psycopg` and `%s` parameterized placeholders.

2 End-to-End Pipeline Architecture

2.1 Architecture Diagram

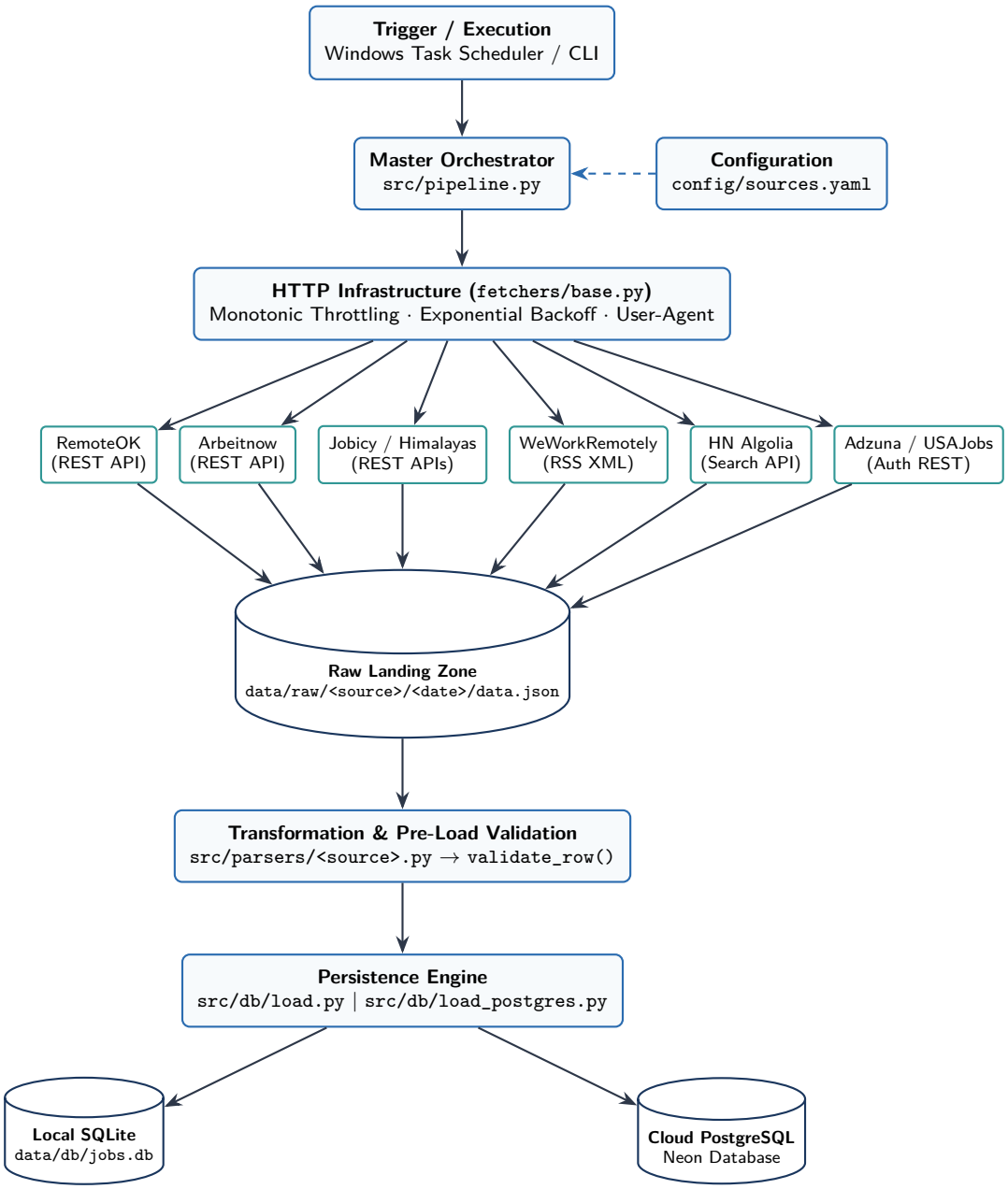


Figure 1: End-to-End Pipeline Architectural Dataflow

2.2 Data Lifecycle Stages

1. **Ingestion Trigger:** Daily execution is triggered via Windows Task Scheduler executing `scripts/run_daily.ps1`. Manual and ad-hoc runs are initiated via `python src/pipeline.py` with optional CLI filters (`-sources`, `-backend`).
2. **Source Extraction:** Network requests are mediated by `src/fetchers/base.py`. Each source is queried using a pull-based REST API call or RSS parse. Rate limits (`rate_limit_sec`) and transparent User-Agent headers are enforced for polite data scraping.

3. **Staging / Raw Persistence:** Before transformation, the raw upstream response payload is saved verbatim to:

   `data/raw/`⟨source⟩/⟨run_date⟩/`data.json`

   This landing zone guarantees full auditability and enables offline parser replays.

4. **Validation & Transformation:** Payloads are mapped by source-specific parsers to the canonical `ROW_FIELDS` contract. Records undergo quality validation (`validate_row()`): required fields (`source`, `external_id`, `retrieved_at`) must be populated, and any `url` present must be syntactically valid with an HTTP/HTTPS scheme and network domain. Malformed records are discarded from the batch and logged.

5. **Loading & Destination:** Validated rows are loaded via `load.py` (SQLite) or `load_postgres.py` (PostgreSQL). Entities are upserted into 3NF normalized tables (`sources`, `companies`, `jobs`, `tags`, `job_tags`). Existing job records matching `(source_id, external_id)` are updated in place, ensuring full idempotency.

3 Code Section & Logic Breakdown

3.1 Logical Section 1: Orchestration & Configuration

- **Related Files:** `src/pipeline.py`, `config/sources.yaml`
- **Detailed Mechanism:** The entry point `main()` parses runtime arguments and reads `sources.yaml`. For every configured source, `run_source()` performs credential checks, invokes extraction, writes the raw snapshot to disk, iterates extracted records through parsers and validators, and executes database upserts. Transactions are committed per source (`conn.commit()`), providing transactional isolation.
- **Key Methods:**
  - `run_source(name, scfg, contact_email, session, run_date, conn, loader) -> dict`: Executes end-to-end ingestion for one source. Returns status dictionary containing metrics (`fetched`, `loaded`, `invalid`).
  - `main()`: Coordinates source iteration, manages backend database connections, and outputs summary execution telemetry.
- **Error Handling & Resilience:** Network failures are trapped within `run_source()`, returning `status: "fetch_error"` without terminating other sources. Missing API credentials safely bypass execution with `status: "skipped_no_key"`.

3.2 Logical Section 2: HTTP Infrastructure & Politeness

- **Related Files:** `src/fetchers/base.py`
- **Detailed Mechanism:** Maintains a monotonic timestamp registry `_last_call_at` per `rate_key`. `throttle()` pauses execution via `time.sleep()` if the elapsed time since the previous call is less than `min_interval_sec`. Wraps HTTP GET in a Tenacity retry handler configured with exponential backoff:

```
@retry(
    stop=stop_after_attempt(4),
    wait=wait_exponential(multiplier=1, min=1, max=10),
    retry=retry_if_exception(_is_retryable),
    reraise=True,
)
def get(session, url, *, rate_key, min_interval_sec, timeout=15, **kwargs):
    throttle(rate_key, min_interval_sec)
    resp = session.get(url, timeout=timeout, **kwargs)
    resp.raise_for_status()
    return resp
```

- **Error Handling & Resilience:** The `_is_retryable()` predicate retries only on HTTP 429 (rate-limited) and 5xx (transient gateway/server errors). Fatal client errors (e.g., 401, 403, 404) fail immediately.

3.3 Logical Section 3: Source Extractors (Fetchers)

- **Related Files:** `src/fetchers/remoteok.py`, `arbeitnow.py`, `jobicy.py`, `himalayas.py`, `wwr.py`, `hn_hiring.py`, `adzuna.py`, `usajobs.py`
- **Detailed Mechanism:** Each module implements `fetch_all(session, cfg)` and `fetch_sample(session, cfg)`.
  - *RemoteOK:* Drops the leading non-job legal/notice item from the raw JSON array.
  - *WeWorkRemotely:* Employs `feedparser` to parse RSS XML feeds into uniform entry dictionaries.
  - *HN Hiring:* Implements a two-step query against the Algolia Search API. Queries `tags=story&query=Who is hiring` to locate the latest monthly parent thread ID, then fetches child comments up to Algolia's limit of 1,000 hits.
  - *Adzuna & USAJobs:* Authenticate via query parameters and custom request headers, flattening deeply nested structures.

3.4 Logical Section 4: Data Normalization & Quality Validation

- **Related Files:** `src/parsers/__init__.py`, and source-specific parsers
- **Canonical Contract:** Every parser instantiates rows conforming to:

```
ROW_FIELDS = (
    "source", "external_id", "title", "company", "location", "remote_type",
    "posted_date", "retrieved_at", "url", "salary_min", "salary_max",
    "description_raw", "tags_raw",
)
```

- **Validation Logic:** `validate_row(row)` ensures `source`, `external_id`, and `retrieved_at` are non-null and checks that any provided `url` satisfies RFC URI standards via `urllib.parse.urlparse`.
- **Zero-Inference Policy:** In accordance with data integrity guidelines, free-text sources (such as Hacker News comments) are captured verbatim in `description_raw` without applying speculative regex heuristics to infer job title or company.

3.5 Logical Section 5: Relational Persistence & Multi-Backend Loading

- **Related Files:** `src/db/load.py`, `load_postgres.py`, `schema.sql`, `schema_postgres.sql`
- **Detailed Mechanism:** On connection, the schema DDL is executed with `CREATE TABLE IF NOT EXISTS`. Rows are loaded through an idempotent normalization sequence:
  1. `get_or_create_source()`: Upserts source name and base URL.
  2. `get_or_create_company()`: Inserts company name (`ON CONFLICT DO NOTHING`).
  3. `upsert_job()`: Inserts job record on composite natural key `UNIQUE(source_id, external_id)`. On conflict, all mutable fields (dates, salary, description) are updated in place, returning `job_id`.
  4. `upsert_tags()`: Deletes existing associations in `job_tags` for `job_id`, inserts new tags into `tags`, and reconstructs the relation.

4 Configuration, Dependencies & Runbook

4.1 Environment Variables & Secrets

Variable	Required	Default	Purpose
`ADZUNA_APP_ID`	Conditional	*None*	Application ID for Adzuna Job Search API.
`ADZUNA_APP_KEY`	Conditional	*None*	Application Key for Adzuna Job Search API.
`USAJOBS_API_KEY`	Conditional	*None*	Authorization Key header for USAJobs API.
`USAJOBS_EMAIL`	Conditional	*None*	User-Agent contact email required by USAJobs.
`DATABASE_URL_UNPOOLED`	Conditional	*None*	PostgreSQL connection string for Neon cloud database (required when `-backend postgres` is set).

Table 2: Required Environment Variables

4.2 External Dependencies

Package	Version	Category	Operational Role
`requests`	≥ 2.31	Network / HTTP	Session pooling and HTTP communication.
`tenacity`	≥ 8.2	Resilience	Exponential backoff and retry decorators.
`PyYAML`	≥ 6.0	Configuration	Parsing `config/sources.yaml`.
`feedparser`	≥ 6.0	RSS / XML	Parsing WeWorkRemotely RSS feeds.
`psycopg[binary]`	≥ 3.1	Database Driver	PostgreSQL client driver for Neon cloud storage.
`python-dotenv`	≥ 1.0	Configuration	Loading environment variables from `.env`.
`pytest`	≥ 7.4	Testing	Automated unit and integration test runner.

Table 3: Third-Party Software Dependencies

4.3 Operational Runbook & CLI Reference

4.3.1 Pre-flight Health Audit

Execute diagnostic checks to verify upstream availability, rate limits, schema stability, and `robots.txt` compliance:

```
python scripts/check_sources.py
```

4.3.2 Standard Local Ingestion (SQLite)

Run full ingestion pipeline across all configured sources into local SQLite:

```
python src/pipeline.py
```

Execute for a subset of sources:

```
python src/pipeline.py --sources remoteok,arbeitnow,jobicy
```

4.3.3 Production Ingestion (PostgreSQL / Neon)

Run ingestion targeting the cloud database:

```
python src/pipeline.py --backend postgres
```

4.3.4 Automated Windows Task Scheduler Execution

The pipeline is invoked daily via PowerShell:

```
powershell -ExecutionPolicy Bypass -File "scripts/run_daily.ps1"
```

4.3.5 Test Suite Execution

Run all test suites including rate-limit timers, retry logic, parsers, and database upsert idempotency:

```
pytest -v
```